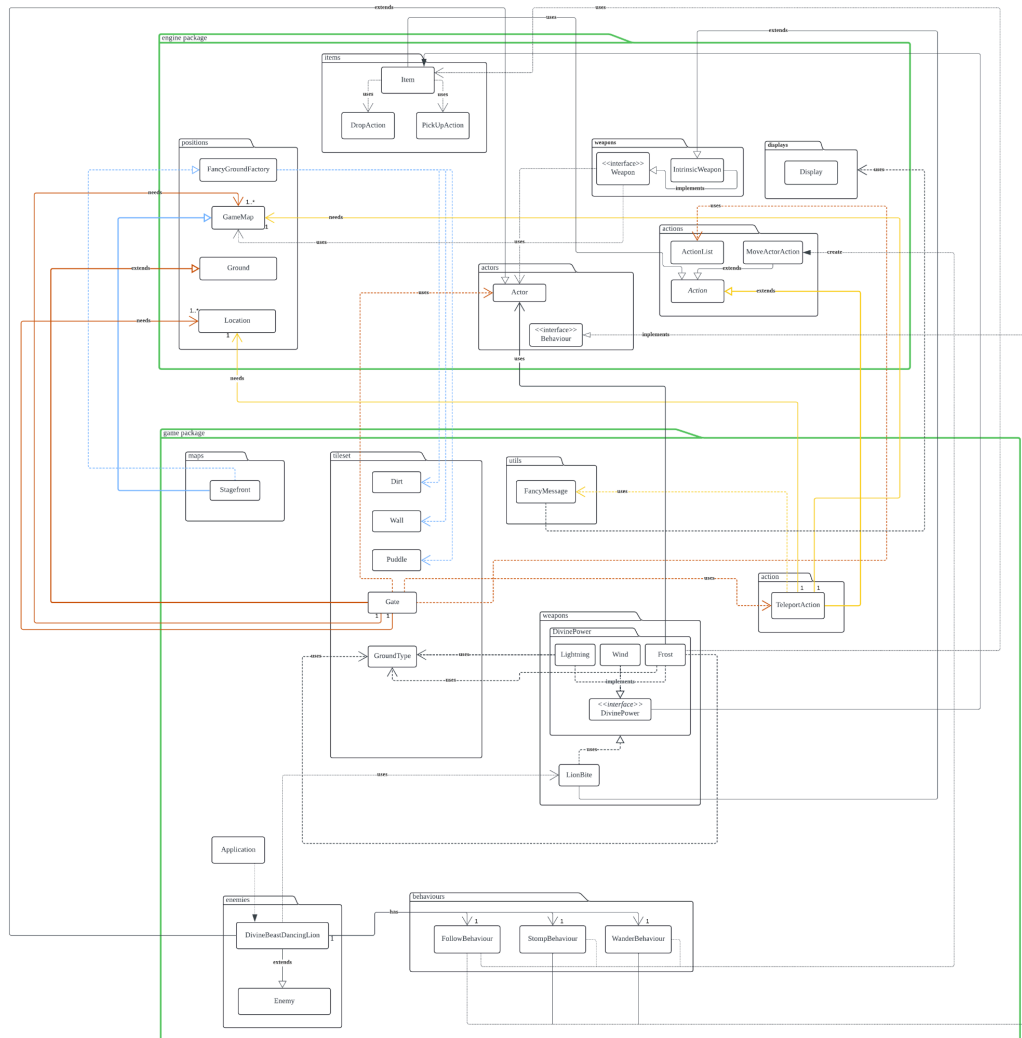


Design Rationale

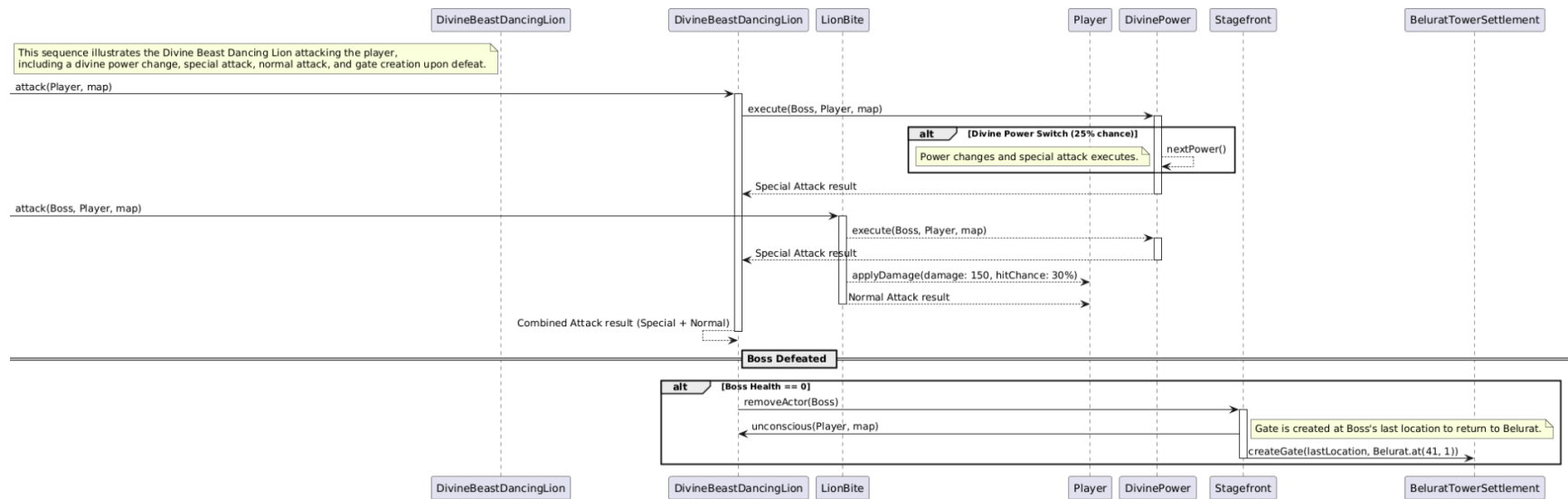
Design Goal

The design aims to break down the system into smaller, more manageable classes to promote modularity and encapsulation. In this system, each class will have a clear and distinct responsibility, making it easier to understand and maintain. By adhering to the SOLID principle and DRY principle, the system should allow for easy extension and promote reusability. This involves identifying common functionalities and abstracting them into reusable components as well as anticipating potential changes to enhance code flexibility and adaptability to changing requirements and future extensions.

REQ1: Divine Beast Dancing Lion



Req 1 UML Diagram



Req 1 Sequence Diagram

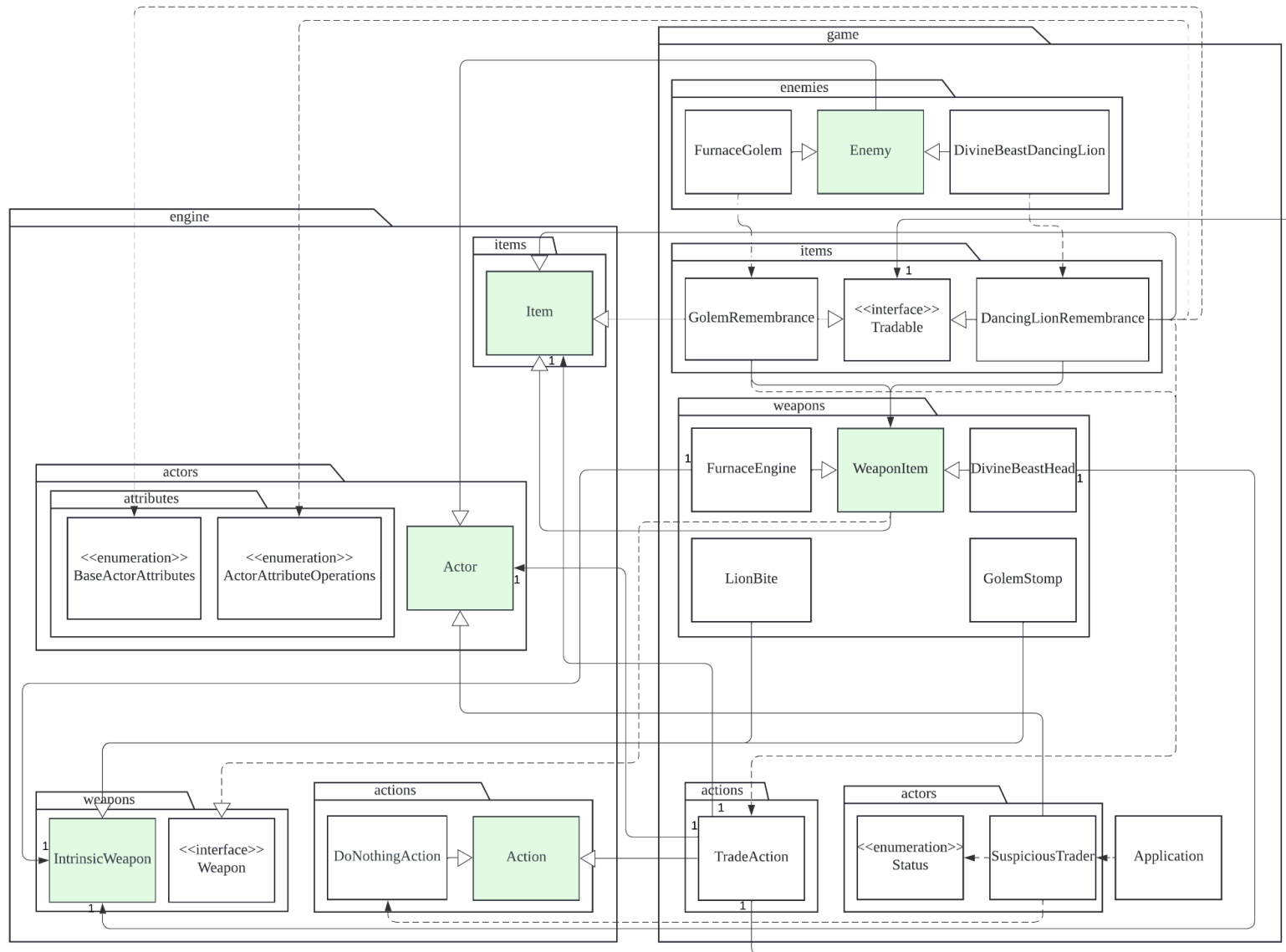
Classes Modified/Created	Roles and Responsibilities	Rationale
DivinePower	Responsible for managing the Trade Action Relationships: - Implemented by classes such as Frost, Wind, and Lightning	The design choices I made for these classes focus on building a modular and extensible structure that will keep the game flexible and ready for future expansions. 1. Starting with the Modular Power System, I used the DivinePower interface along with its implementations (Frost, Wind, and Lightning) to allow each power to operate independently while following a consistent structure. This setup aligns with the Open-Closed Principle, as it lets me add new powers without changing existing code. The ability to switch between powers brings depth to the boss fights, creating a more dynamic experience. 2. For Weapon Specialization, I chose to extend IntrinsicWeapon with LionBite so that the Divine Beast Dancing Lion could directly utilize divine powers during its attacks. By encapsulating this specialized behavior within LionBite, I keep the attack mechanics contained and organized. This makes the LionBite class reusable and simplifies the addition of similar attack types in the future. 3. The Dynamic Boss Behavior in DivineBeastDancingLion is another important aspect. Using a behavior-driven approach, I incorporated actions like attack, follow, and wander through a composition of behavior classes (like WanderBehaviour and AttackBehaviour). This setup gives the boss complex, multi-faceted AI, allowing for more engaging encounters. It also makes the AI adaptable to future requirements with minimal modifications.
Frost	Implements the DivinePower interface, representing a specific divine power that causes the target to drop items if they are on water. Relationships: Implements DivinePower - Has a 1:1 association with Wind for transitioning to the next power - Uses Actor, GameMap, Location, Ground, Item, and GroundType classes	
Lightning	Implements the DivinePower interface, representing a divine power that strikes surrounding actors with lightning, causing additional damage if the ground has a water capability. Relationships: - Implements DivinePower - Has associations with Frost and Wind for transitioning to the next power - Uses Actor, GameMap, Location, Exit, Ground, and GroundType classes	

Wind	Implements the DivinePower interface, representing a divine power that strikes surrounding actors with lightning, causing additional damage if the ground has a water capability. Relationships: - Implements DivinePower - Has associations with Frost and Wind for transitioning to the next power - Uses Actor, GameMap, Location, Exit, Ground, and GroundType classes	4. In terms of Game Setup and Progression, the Application class handles all initialization, from maps and characters to progression mechanics like gate creation upon the boss's defeat. Centralizing these elements in Application not only keeps setup organized but also makes it easier to add or modify levels and progression paths. It adheres to the Single Responsibility Principle, separating the game setup from gameplay logic and keeping things clean. 5. With Interactive Display, I added FancyMessage to control ASCII art for game events and titles, adding visual flair that enhances immersion. By consolidating these messages in a utility class, it's simple to update or reuse them as needed, keeping display effects cohesive throughout the game. 6. Lastly, the Context-Sensitive Environment is represented through GroundType, an enum for environmental resistances like water resistance. This detail allows certain powers, such as Frost and Lightning, to interact with the environment, adding another layer to gameplay. Using an enum for ground types keeps things straightforward, making it easy to add new types without cluttering the code. Overall, this approach results in a highly interactive, adaptable environment that keeps code modular and extensible. By designing with clear boundaries and organized responsibilities, I've aimed for a flexible structure that can accommodate new features and gameplay mechanics with minimal disruption to the existing codebase.
LionBite	Extends IntrinsicWeapon to represent a specific attack (bite) for the Divine Beast Dancing Lion, capable of transitioning between different divine powers. Relationships: - Extends IntrinsicWeapon - Has a 1:1 association with DivinePower - Uses Actor, GameMap, and Wind classes	
DivineBeastDancingLion	Extends Enemy to represent a boss enemy with unique behaviours, including attack, follow, and wander actions. When defeated, it opens a gate to a specified location. Relationships:	

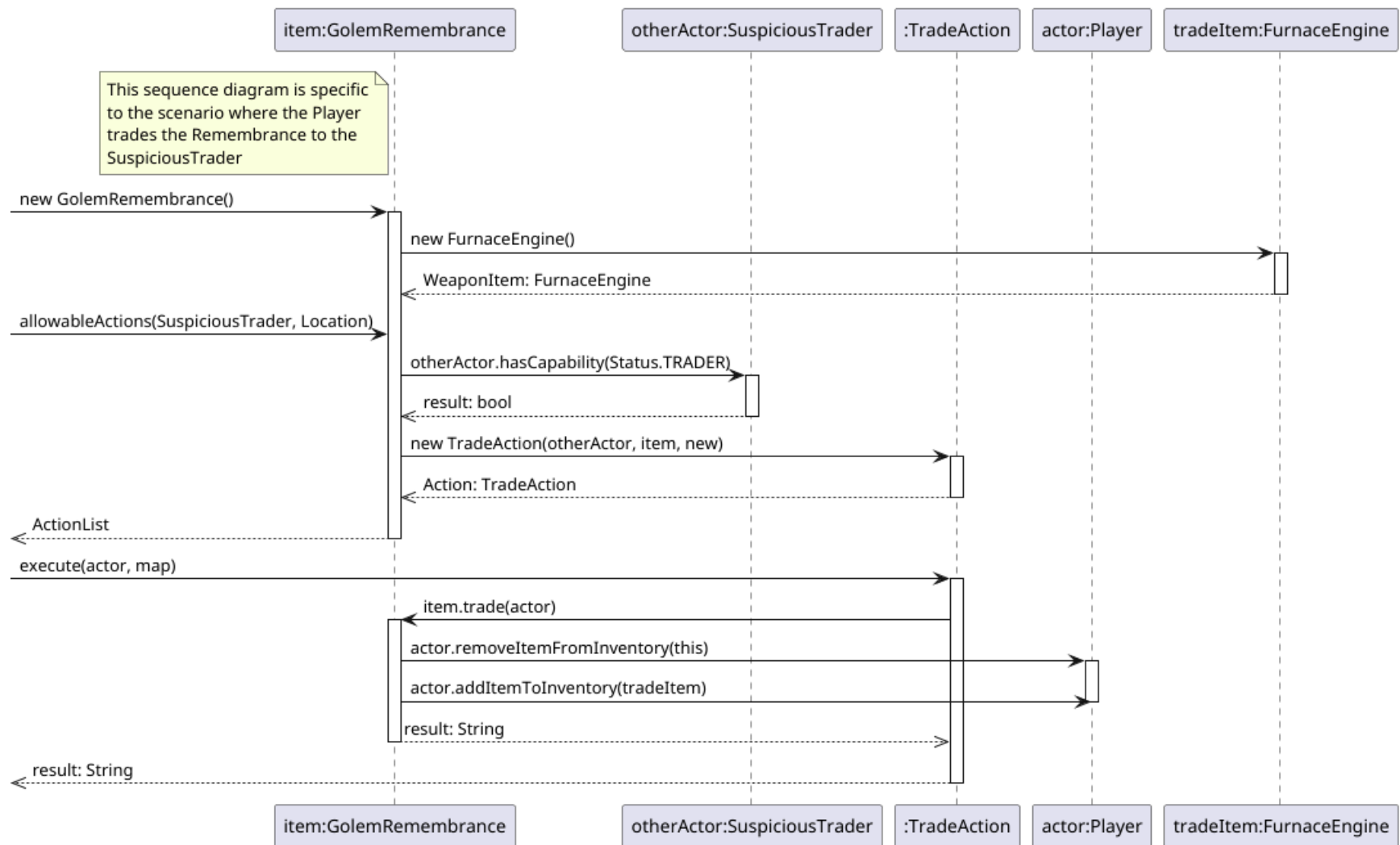
	- Extends Enemy - Uses GameMap, LionBite, and Wind classes - Depends on WanderBehaviour, AttackBehaviour, and FollowBehaviour for behavior - Uses Gate and Location	
GroundType	Defines an enumeration to represent different types of ground resistances, such as being water-resistant. Relationships: - Used by classes like Frost and Lightning to check ground resistance capabilities	
FancyMessage	Provides fancy ASCII messages for various game events and locations, along with a method to print these messages with a delay effect for dramatic effect and move from Relationships: - Uses Display for printing messages - Uses TreeMap to map locations and events to specific messages - Contains static fields for message, STAGEFRONT	
Application	Responsible for setting up and starting the game by initializing the world, maps, players, enemies, items, gates, and spawners.	

	Relationships: - Uses World, Display, FancyMessage, and various game components such as Player, DivineBeastDancingLion, Graveyard, and Gate - Interacts with BeluratTowerSettlement and Stagefront maps for gate setup and boss placement	
--	--	--

REQ2: Remembrance of a Legend



Req 2 UML Diagram



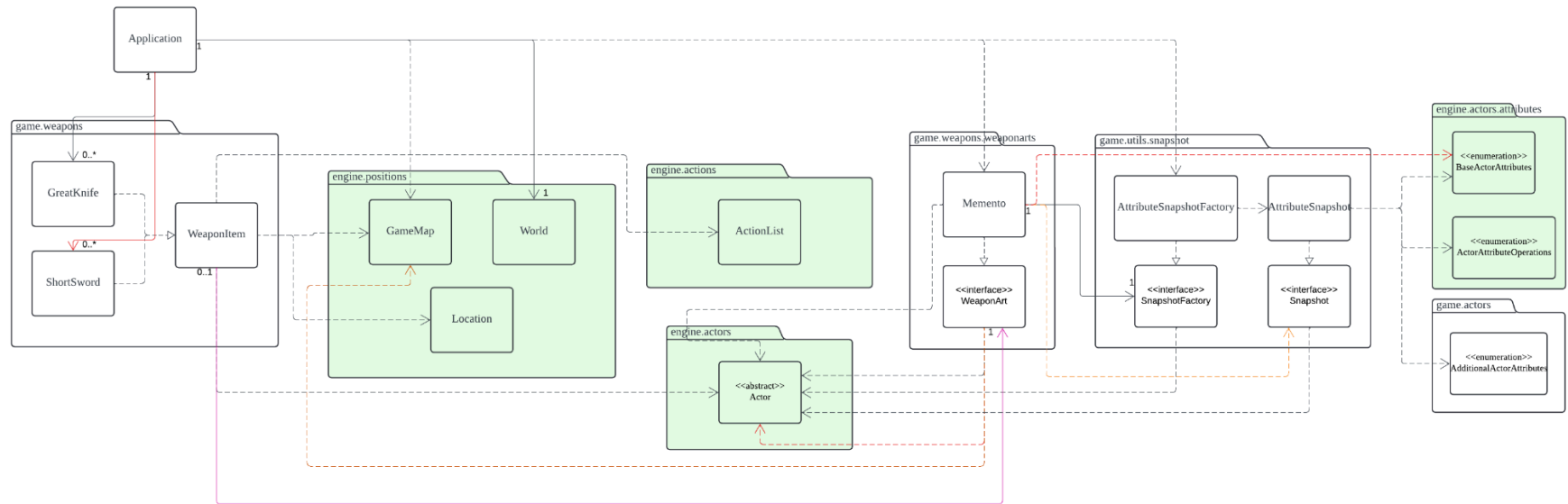
Req 2 Sequence Diagram

Classes Modified/Created	Roles and Responsibilities	Rationale
TradeAction	Responsible for managing the Trade Action Relationships: • Extends Action abstract class • Has a 1:1 association with Actor • Has a 1:1 association with Item • Has a 1:1 association with Tradable	Alternate Solution: Have each remembrance be associated with the concrete implementation instead of the abstract class IntrinsicWeapon. This violates the OCP as changes to the LionBite or GolemStomp would probably necessitate changes to the respective DivineBeastHead and FurnaceEngine. It would also violate DIP as now they depend on concrete implementations instead of abstractions which could lead to a Connascence of Type due to the deep coupling between the two classes. Finalised solution: Creation of a new Action: TradeAction to perform trading Creation of a new Actor: SuspiciousTrader for the Player to trade with Creation of new WeaponItems: FurnaceEngine and DivineBeastHead Creation of new Items: DancingLionRemembrance and GolemRemembrance Modification of existing classes: FurnaceGolem and DivineBeastDancingLion's unconscious() had to be modified in order to have them drop the Remembrances upon death Creation of a new Status: TRADER Reasons for Decision: 1. The TradeAction manages the trade process, including managing how the trade is displayed to the player in the menu and executing it. This ensures that only trading logic is managed here, keeping it focused. (<i>Single Responsibility Principle</i>) 2. The creation of the Tradable interface allows for new items to be tradable through implementing the interface. This enables DancingLionRemembrance and GolemRemembrance and any potentially tradable Items to
SuspiciousTrader	Responsible for managing the SuspiciousTrader NPC Relationships: • Extends Actor • Depends on Status	
DivineBeastHead	Responsible for representing an item that gives the Player the abilities of the Dancing Lion Relationships: • Extends WeaponItem • Has a 1:1 association with IntrinsicWeapon	
FurnaceEngine	Responsible for representing an item that gives the Player the abilities of the Furnace Golem Relationships: • Extends WeaponItem • Has a 1:1 association with IntrinsicWeapon	

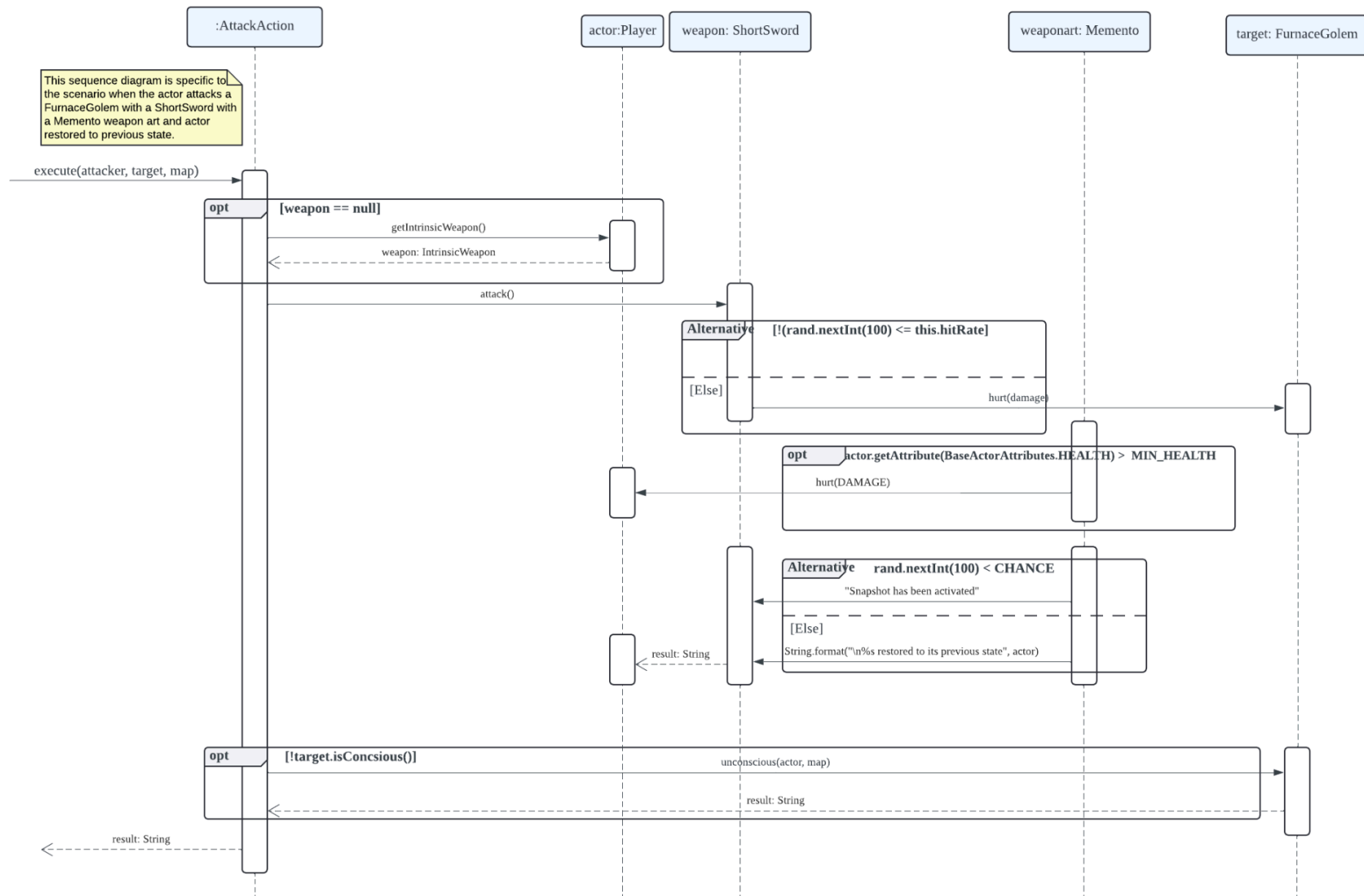
DancingLionRemembrance	Responsible for the item Remembrance of the Dancing Lion Relationships: • Extends Item • Implements Tradable • Has a 1:1 association with WeaponItem • Has a dependency on ActorAttributeOperations and BaseActorAttributes • Has a dependency on TradeAction	support trading while leaving the base TradeAction implementation untouched. (<i>Open Closed Principle</i>) - Both DancingLionRemembrance and GolemRemembrance implement the Tradable interface. This means they can be traded via TradeAction as Tradable items, and TradeAction does not need to distinguish between them. (<i>Liskov substitution Principle</i>) - The Tradable interface focuses only on defining the behaviour required for trading items. Only items that need to be traded implement Tradable (e.g., DancingLionRemembrance and GolemRemembrance). This prevents other non-tradable items from implementing unnecessary methods. (<i>Interface Segregation Principle</i>) - Tradable Dependency in TradeAction: TradeAction relies on the Tradable interface rather than specific item types. This allows any item implementing Tradable to be processed by TradeAction without requiring knowledge of concrete classes like DancingLionRemembrance or GolemRemembrance. (<i>Dependency Inversion Principle</i>) Limitations and Tradeoffs: • Connascence of Name: Tradables and SuspiciousTrader must agree on the enum TRADER being what denotes whether the TradeAction should appear in the menu. •
GolemRemembrance	Responsible for the item Remembrance of the Furnace Golem Relationships: • Extends Item • Implements Tradable • Has a 1:1 association with WeaponItem • Has a dependency on TradeAction	
Tradable	An interface that is responsible for managing behaviours for Items that are Tradable Relationships: • Has a 1:1 association with Actor	
FurnaceGolem	Responsible for Furnace Golem behaviour Relationships:	

	• Extends Enemy • Has a dependency on GolemRemembrance	
DivineBeastDancingLion	Responsible for DivineBeastDancingLion behaviour Relationships: • Extends Enemy • Has a dependency on LionRemembrance	
Application	Class that is responsible for starting the game. Relationships: • It depends on SuspiciousTrader	

REQ3: Memento



Req 3 UML Diagram

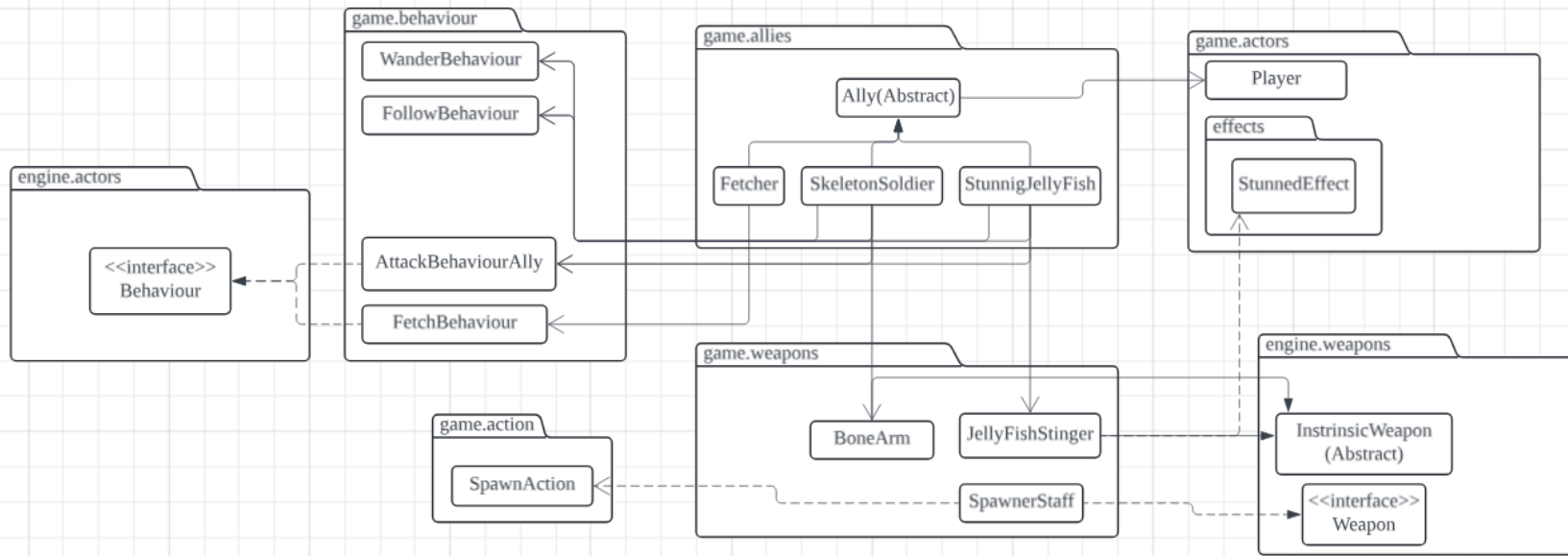


Req 3 Sequence Diagram

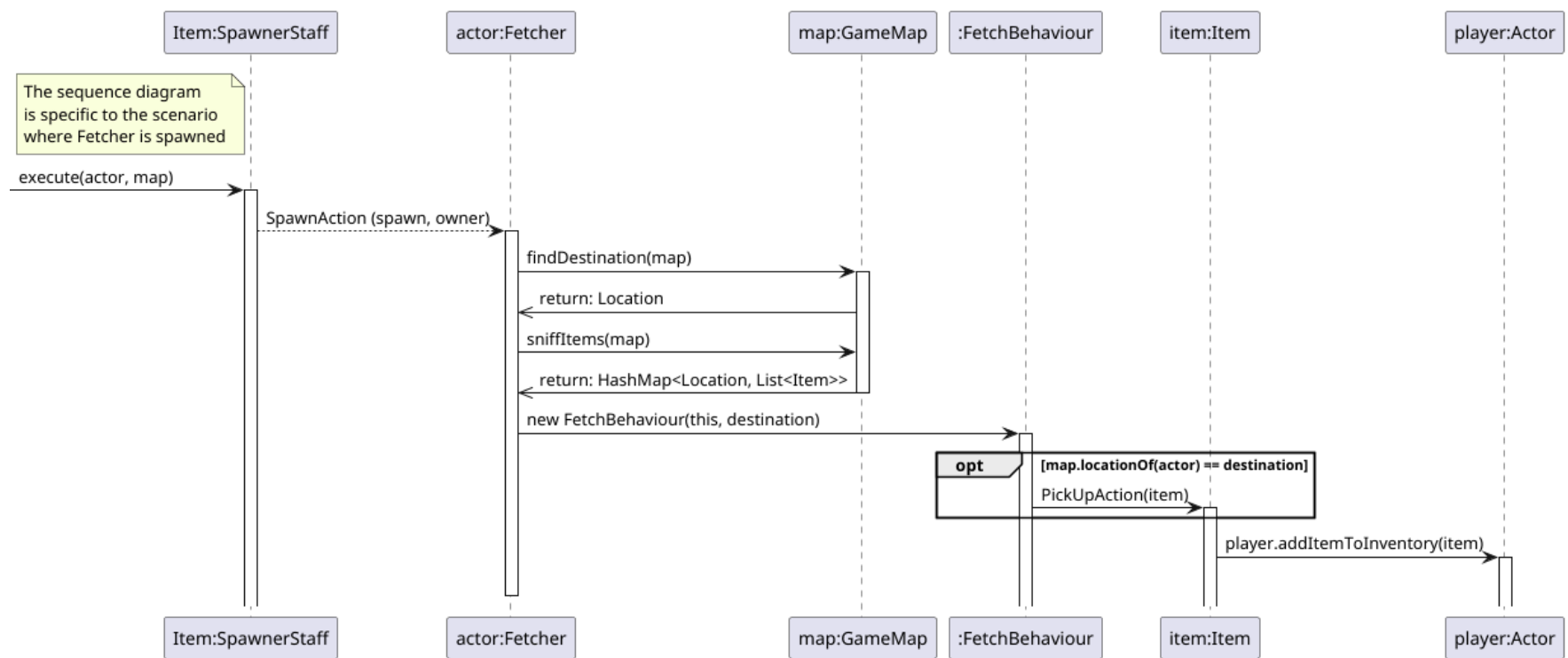
Classes Modified/Created	Roles and Responsibilities	Rationale
Snapshot	Interface for snapshot Relationship: 1. It depends on the Actor class to determine which actor is affected by the snapshot.	Alternate Solution: Instead of using a factory pattern design, let the Memento class handle all types of snapshots (e.g., AttributeSnapshot, LocationSnapshot) by hardcoding using multiple if-else statements. The memento class will have multiple responsibilities including creating, storing, and restoring snapshots. Finalised solution: Create a new Memento class and implement the WeaponArt interface. Define the execute function of the memento weapon art in its class. Implemented the factory method design pattern to tell the memento class what to snapshot. Use stack to store snapshot objects, meaning that snapshot objects has to be created first before using it(Connascence of Execution). Reasons for Decision: 1. The Snapshot objects will be responsible in saving and restoring different states and a SnapshotFactory object will handle the instantiation of the Snapshot objects. Each of these classes have a single well-defined responsibility which leads to low coupling and better code maintainability. <i>(Single Responsibility Principle)</i> 2. The Snapshot interface allows the code to be open for extension but closed for modification. For example, if another type of snapshot such as location snapshot were to be created, a new class could be created implementing the Snapshot interface without modifying any of the existing code. This reduces the risk of introducing new bugs and breaking the existing code. <i>(Open Closed Principle)</i>
AttributeSnapshot (implements Snapshot)	Class representing the snapshot of an actor's attributes. Relationships: 1. It implements the Snapshot class to achieve snapshot functionality. 2. It depends on the Actor class to determine which actor is affected by the snapshot. 3. It depends on BaseActorAttributes to access the actor's, health and mana. 4. It depends on AdditionalActorAttributes to access the actor's strength. 5. It depends on ActorAttributeOperations to update the actor's attributes.	
SnapshotFactory	Factory interface for snapshots. Relationships: 1. It depends on the Actor class to access the actor's attributes. 2. It depends on the Snapshot interface to create snapshots.	

AttributeSnapshotFactory (implement SnapshotFactory)	Class representing the AttributeSnapshot factory. Relationships: 1. It depends on the Actor class to access the actor's attributes. 2. It depends on the Snapshot interface to create AttributeSnapshot.	3. Different types of snapshots can be used interchangeably whenever an instance of Snapshot is expected without breaking the unexpected behaviour. (<i>Liskov substitution Principle</i>) 4. The snapshot interface allows its subclasses to only depend on its methods. The interface is also small and focused, with only the restore() function being defined, ensuring that each snapshot only implements what is relevant to it. (<i>Interface Segregation Principle</i>) 5. The Memento depends on the Snapshot interface. By doing so, multiple if-else statements and instanceof checks can be avoided. This means that the Memento class does not need to depend on every type of snapshot, making it more maintainable as it does not depend on low-level modules. (<i>Dependency Inversion Principle</i>) Limitations and Tradeoffs: 1. Introducing new types of snapshots means creating more classes. This is because we'll need to create the snapshot class itself as well as its factory to return the snapshot object itself which can cause even greater complexities and make it difficult to get the flow of the implementation logic. 2. The current implementation of the Snapshot interface haven limited extensibility. Currently, the snapshot interface can only snapshot either the attacker or target and not both.
Memento (implements WeaponArt)	Class representing Memento weapon art. Relationships: 1. It implements the WeaponArt interface to achieve weapon art functionality. 2. It depends on SnapshotFactory to determine what to snapshot. 3. It depends on the Snapshot interface to store a stack of Snapshot objects. 1. It depends on the Actor class to determine which Actor is activating the weapon art.	
Application	The class is responsible for starting the game. Relationships: 1. It depends on the GreatKnife class to instantiate the GreatKnife object. 2. It depends on the ShortSword class to instantiate the ShortSword object. 3. It depends on the World class to run the whole program.	

REQ4: Creative Mode



Req 4 UML Diagram



Req 4 Sequence Diagram

Classes Modified/Created	Roles and Responsibilities	Rationale
SpawnerStaff	A new Item that represents a Staff that can spawn allies Relationships: • Extends Item • Has a dependency on SpawnAction • Has a dependency on SkeletonSoldier, StunningJellyfish, and Fetcher	Alternate Solution: The items can have location built in to make fetcher easier to locate the items. Other than that, the spawner staff can be separated into three different weapons to allow the staff to spawn one ally each. Finalised Solution: The final solution made it so Fetcher class has to handle the locating and locking into the item location. It searches the whole map then targets the closest item and moves towards it to return the item to the player. Next, one SpawnerStaff can spawn all the allies and the player is able to choose who to spawn. SOLID Principle: 1. To make sure that each class only has one responsibility each JellyfishStinger and StunnedEffect are two different classes (SRP) 2. To allow extension but less modification, we add STUNNED into StatusEffect to allow additional statuses in the future(OCP) 3. All classes accept all Actor types which is the base type for both allies and enemies alike. (LSP) 4. The Weapon interface is not forced upon Intrinsic Weapons such as JellyfishStinger (ISP) 5. Ally abstract class only works as a blueprint, further modifications are overriding the current methods however, Ally does not call any of its children methods whereas its children will only depend on Ally (DIP) Limitations: The one limitation is that in order to add more spawnable allies, there needs to be editing of SpawnerStaff class which violates the SOLID principle. Other than that, the player currently has no ability to command whatever the allies want to do and allies will
Ally	An abstract Actor that represents an Ally Relationships: • Extends Actor	
StunningJellyfish	A new Ally that represents the StunningJellyfish Relationships: • Extends Ally • Has a dependency on JellyfishStinger • Has a dependency on AttackBehaviourAlly, FollowBehaviour, WanderBehaviour	
JellyfishStinger	A new IntrinsicWeapon that represents the StunningJellyfish's IntrinsicWeapon Relationships: • Extends IntrinsicWeapon • Has a dependency on Status • Has a dependency on StunnedEffect	

SkeletonSoldier	A new Ally that represents the SkeletonSoldier Relationships: • Extends Ally • Has a dependency on BoneArm • Has a dependency on AttackBehaviourAlly, FollowBehaviour, WanderBehaviour	do only as their behaviour allows them to do.
BoneArm	A new IntrinsicWeapon that represents the SkeletonSoldier's IntrinsicWeapon Relationships: • Extends IntrinsicWeapon	
Fetcher	A new Ally that represents the Fetcher Relationships: • Extends Ally • Has a 1:1 association with GameMap • Has a 1:1 association with Location	
Player	An Actor that represents the Tarnished Relationships: • Extends Actor	
Enemy	An abstract Actor that represents an Enemy Relationships: • Extends Actor • Has a dependency on Status	
SpawnAction	A new Action that represents the action of	

	the Player spawning an Ally Relationships: • Extends Action • Has a 2:1 association with Actor • Has a dependency on RandomNumberGenerator	
Status	Enum that has all Actor Statuses	
StunnedEffect	Class that handles the stunned enemies Relationships: • Extends StatusEffect	
DeathAction	An action that represents the death of an Actor Relationships: • Extends Action	
Application	The class is responsible for starting the game. Relationships: • Has a dependency on SpawnerStaff	